

WEEK 10 — Unit 4: Maven Build Automation

Topic: Why Build Tools Exist, Project Object Model (POM), Directory Structure & Build Lifecycle Phases
Course: INT 332 – DevOps

Note 1: Why Build Tools Exist and Problems Solved by Automated Builds

The Problem Before Build Tools

In the early days of Java development, building a project meant manually performing a series of steps every single time a change was made:

- Compiling each .java source file using javac
- Managing the classpath manually — specifying every .jar dependency file location
- Running tests manually one by one
- Packaging the compiled classes into a .jar or .war file manually using jar command
- Copying the output to the right deployment directory

For a small project with two or three files, this was manageable. But as projects grew to hundreds of source files with dozens of external libraries, doing all of this manually became completely impractical. A single developer could spend hours just setting up and building a project on a new machine.

Problems Solved by Automated Build Tools

1. Dependency Management Real-world Java applications depend on dozens of external libraries — logging frameworks, database drivers, HTTP clients, testing libraries, and more. Before build tools, developers had to manually download every .jar file, keep track of the correct version, and ensure every team member had the same versions. Build tools automate this entirely — you declare what you need, and the tool downloads it automatically from a central repository.

2. Reproducible Builds A build tool ensures that the same source code always produces the same output regardless of who builds it or on which machine. This is critical in team environments and CI/CD pipelines where builds must be consistent.

3. Standardised Project Structure Without a standard structure, every project looked different. One developer might put source files in src/, another in source/, another in app/java/. Build tools enforce a convention — everyone follows the same layout, making it immediately familiar to any developer who joins the project.

4. Lifecycle Automation A build is not just compilation — it involves validation, compilation, testing, packaging, and deployment. Build tools define a standard lifecycle and execute all these steps in the correct order with a single command.

5. Plugin Ecosystem Build tools support plugins that extend their functionality — generating documentation, running code quality checks, creating Docker images, publishing to artifact registries, and much more.

6. Transitive Dependency Resolution If library A depends on library B which depends on library C, you only need to declare A — the build tool figures out and downloads B and C automatically.

What is Maven?

Apache Maven is a build automation and project management tool primarily used for Java projects. It was created to address all the problems listed above. Maven follows a "convention over configuration" philosophy — it provides sensible defaults so that developers can build a project with minimal configuration, while still allowing customisation when needed.

Maven uses an XML file called pom.xml (Project Object Model) to describe the project, its dependencies, and how it should be built.

Core Maven Commands:

- mvn compile — compiles the source code
 - mvn test — runs unit tests
 - mvn package — packages the compiled code into a .jar or .war
 - mvn install — installs the package into the local repository
 - mvn clean — deletes the target/ directory (compiled output)
 - mvn clean install — cleans and then runs the full build lifecycle
-